



AIN SHAMS UNIVERSITY FACULTY OF ENGINEERING

CSE211s: Introduction to Embedded Systems

Project Technical Report

Section 1_Team 11

Name	ID
Hassan Khaled Sheble	2101998
Mohamed Sameh Zahran	2201395
David Safwat Samuel	2200770
Nourhan Mahmoud Saleh	2200185
Heba Rabea Teleb	2200428

Submitted to:

Dr/Mohamed El-Shafey

Eng/ Ayman Baharia

Eng/ Abdallah Mohamed

Table of Contents

Abstract	2
1. Introduction	2
2. Project Objectives	3
3. System Requirements	3
3.1 Functional Requirements	3
3.2 Non-Functional Requirements	3
4. Hardware Components	4
5. System Architecture	4
6. Working Principle	5
7. Software Structure.....	6
7.1 Startup Code.....	6
7.2 Main Program Logic.....	7
7.3 Interrupt Service Routines (ISRs)	11
8. Implementation.....	12
9. Results and Testing.....	13
10. Advantages of the System	14
11. Future Improvements	14
12. Conclusion	14
13. GitHub Repository	14

Abstract

This project presents the design and implementation of an embedded robotic system capable of **following an object using an ultrasonic sensor and a Nucleo development board**. The system continuously measures the distance between the robot and a moving obstacle using the HC-SR04 ultrasonic sensor. According to the measured distance, the robot moves forward, or stops to maintain a safe following distance.

The project combines embedded programming, real-time sensing, motor control, and modular software design. The developed system demonstrates the practical application of embedded systems in autonomous robotic platforms. The implementation includes startup initialization, interrupt-based timing techniques, motor driver interfacing, and distance-based decision making.

1. Introduction

Embedded systems are widely used in modern robotic applications because of their ability to perform dedicated tasks efficiently and in real time. Autonomous robotic vehicles rely on sensors and microcontrollers to interact with the surrounding environment and make intelligent decisions.

The aim of this project is to design a robot car capable of following an object moving in a straight line while maintaining a safe distance. The system uses an HC-SR04 ultrasonic sensor connected to a Nucleo development board to measure the distance between the robot and the target object. Based on the measured distance, the robot adjusts its movement automatically.

- The project demonstrates important embedded systems concepts such as:

- Sensor interfacing
- Real-time processing
- Motor control
- Interrupt handling
- Modular software structure

2. Project Objectives

The main objectives of this project are:

- Measure distance using an ultrasonic sensor
- Control the movement of the robot car automatically
- Maintain a suitable distance from the obstacle
- Stop safely if the obstacle stops
- Develop modular and readable embedded software
- Utilize hardware peripherals efficiently

3. System Requirements

3.1 Functional Requirements

- The system should:

- Detect the distance between the robot and the object
- Move forward when the object is far away
- Stop when the object is within the safe distance
- Continuously update measurements in real time

3.2 Non-Functional Requirements

- The system should:

- Respond quickly to object movement
- Provide stable and smooth operation
- Maintain accurate distance measurement
- Use modular and maintainable code structure

4. Hardware Components

Component	Function
STM32 Nucleo Board	Main processing unit
HC-SR04 Ultrasonic Sensor	Distance measurement
L298N Motor Driver	Motor control
DC Motors	Robot movement
Robot Chassis	Mechanical platform
Wheels	Mobility
Battery Pack	Power supply
Jumper Wires	Electrical connections

The project used the STM32 Nucleo development board as the main controller due to its efficient GPIO handling, timer modules, and interrupt support capabilities. The ultrasonic sensor was mounted at the front side of the robot to continuously detect nearby objects.

5. System Architecture

The developed system consists of four major subsystems:

- Ultrasonic sensing subsystem
- Embedded processing subsystem
- Decision-making subsystem
- Motor control subsystem

The HC-SR04 ultrasonic sensor continuously measures the distance between the robot and the obstacle. The measured data is processed by the STM32 Nucleo board, which determines the appropriate movement action based on predefined distance thresholds. The motor driver module then controls the DC motors to move the robot forward, or stop.

The system operates in real time and continuously updates the movement behavior according to the detected distance.

6. Working Principle

The robot follows an object by continuously measuring the distance using the HC-SR04 ultrasonic sensor. The sensor transmits ultrasonic waves and measures the echo return time from the obstacle.

The measured distance is calculated using:

$$d = \frac{v \times t}{2}$$

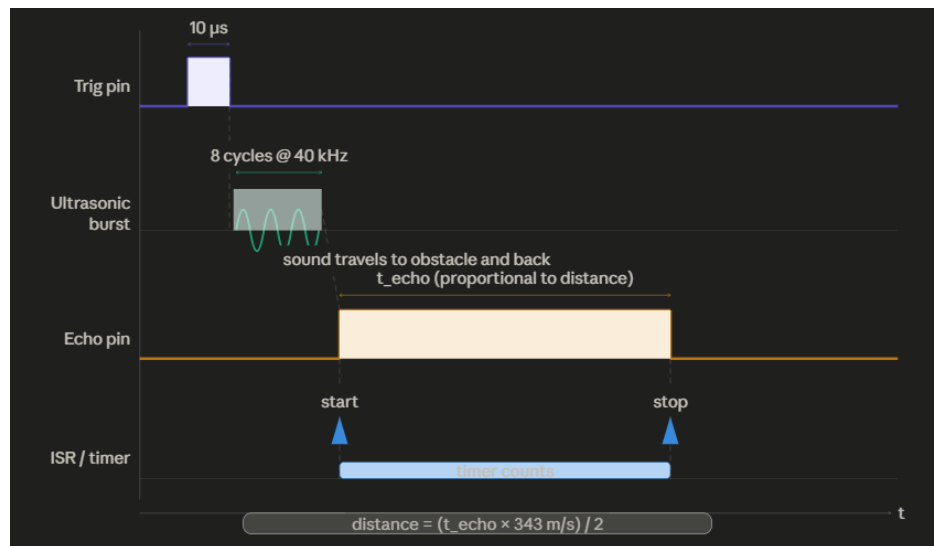
Where:

- d = measured distance
- v = speed of sound
- t = echo pulse duration

The embedded software compares the measured distance with predefined threshold values:

- If the obstacle is far away, the robot moves forward.
- If the obstacle is within the safe distance, the robot stops.

This logic allows the robot to maintain a safe following distance automatically.



7. Software Structure

The software was designed using modular programming concepts to improve readability and simplify debugging and maintenance.

- The program is divided into the following sections:

- Initialization functions
- Distance measurement functions
- Motor control functions
- Decision-making logic
- Interrupt handling routines

The software continuously executes inside the main loop while timer-based operations and sensor timing are handled using interrupts for higher accuracy.

7.1 Startup Code

The startup stage of the software is responsible for initializing all hardware peripherals required for the robot operation before entering the main execution loop. During initialization, the ultrasonic sensor pins, motor driver pins, PWM outputs, and interrupt callbacks are configured.

The project follows an object-oriented programming approach by creating separate classes for ultrasonic sensing and motor control. The constructors of these classes automatically configure the required peripherals during object creation.

The **RobotDrive** constructor **initializes the PWM outputs used for motor speed control and sets the PWM period for stable motor operation.** The **Ultrasonic** constructor **configures the trigger and echo pins and attaches interrupt callback functions to the echo pin.**

- The following code initializes interrupt detection for the ultrasonic sensor:

```
// Attach interrupt when echo signal rises
_echo.rise(callback(this, &Ultrasonic::onRise));

// Attach interrupt when echo signal falls
_echo.fall(callback(this, &Ultrasonic::onFall));
```

Code Explanation

- `rise()` detects the beginning of the echo pulse.
- `fall()` detects the end of the echo pulse.
- `callback()` links the interrupt event to the corresponding ISR function.

- The following code initializes the PWM frequency for both motors:

```
// Set PWM period for left motor
_lpwm.period(0.001f);

// Set PWM period for right motor
_rpwm.period(0.001f);
```

Code Explanation

- PWM is used to control motor speed.
- The PWM period is set to 1 ms for stable motor operation.
- Both left and right motors are initialized during startup.

This initialization stage ensures that all peripherals operate correctly before the robot starts autonomous movement.

7.2 Main Program Logic

The main program continuously executes inside an infinite loop to allow real-time robot operation. The robot periodically measures the distance between itself and the obstacle using the HC-SR04 ultrasonic sensor and determines the appropriate movement action based on the measured distance.

- At the beginning of each loop iteration, the ultrasonic sensor is triggered to transmit ultrasonic waves.

```
// Trigger ultrasonic sensor
sensor.trigger();
```

Code Explanation

- Sends a trigger pulse to the HC-SR04 sensor.
- Starts the ultrasonic measurement process.

- After triggering the sensor, the program waits briefly to allow the interrupt routines to calculate the distance value.

```
// Wait for ISR to calculate distance  
ThisThread::sleep_for(10ms);
```

- The measured distance is then read from the ultrasonic module.

```
// Read measured distance  
float currentDist = sensor.getDistance();
```

- The robot continuously prints the measured distance to the serial monitor for debugging and testing purposes.

```
// Print distance on serial monitor  
printf("Distance: %d cm\r\n", (int)currentDist);
```

- The movement decision is made using conditional statements based on predefined distance thresholds.

```
// Stop if obstacle is too close or sensor reading is invalid  
if (currentDist < DIST_SAFE || currentDist > 400.0f) {  
    robot.stop();  
}
```

Code Explanation

- Stops the robot if the obstacle becomes dangerously close.
- Stops the robot if the sensor reading exceeds the valid ultrasonic range.

- If the object is far away, the robot moves forward at higher speed.

```
// Move forward at higher speed  
else if (currentDist > 50.0f) {  
    robot.drive(SPEED_FAR);  
}
```

- If the object is within medium range, the robot moves at lower speed for smoother operation.

```
// Move forward slowly near the target  
else if (currentDist > DIST_TARGET) {  
  robot.drive(SPEED_NEAR);  
}
```

- Otherwise, the robot stops.

```
// Stop at target distance  
else {  
  robot.stop();  
}
```

- The loop repeats continuously every 100 ms to ensure stable and responsive object-following behavior.

```
// Delay before next loop iteration  
ThisThread::sleep_for(90ms);
```

- **The implemented logic allows the robot to:**

- Follow moving objects
- Maintain safe distance
- Prevent collisions
- Operate autonomously in real time

PROGRAM FLOWCHART

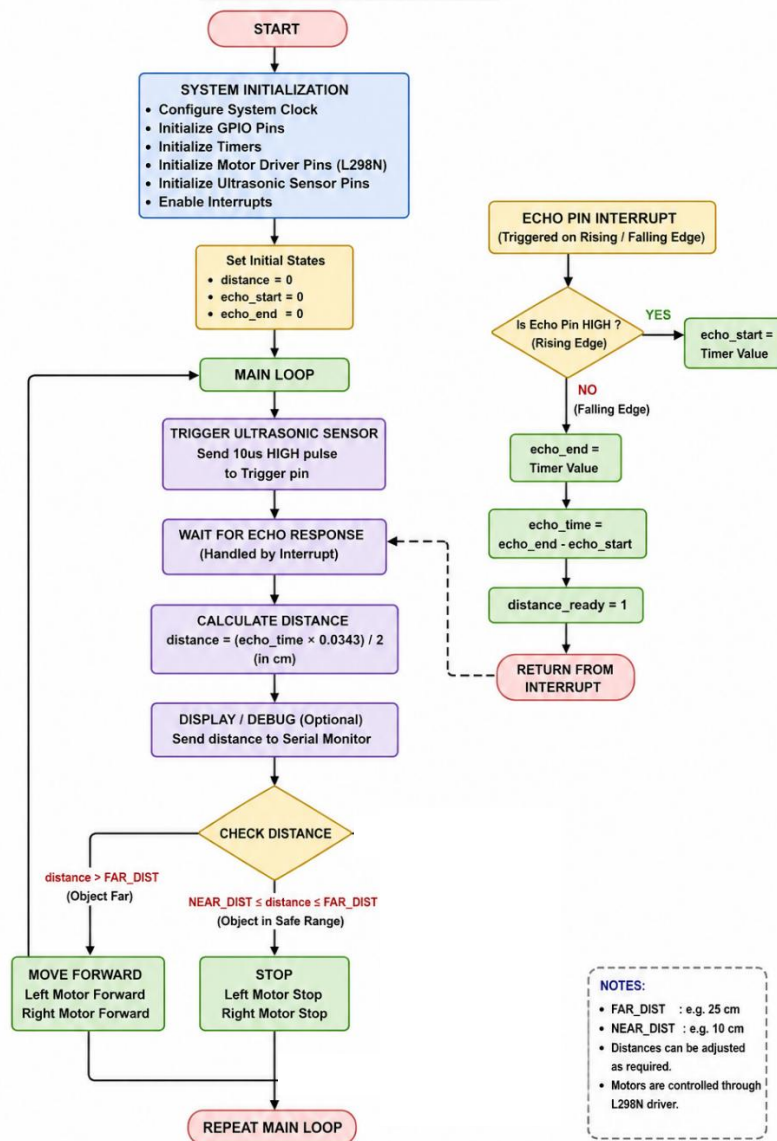


Figure 1: Program Flowchart

7.3 Interrupt Service Routines (ISRs)

Interrupt Service Routines (ISRs) were implemented to improve the accuracy of ultrasonic distance measurement. The HC-SR04 ultrasonic sensor generates an echo pulse whose duration corresponds to the distance between the robot and the obstacle.

Instead of continuously polling the echo pin, interrupts are used to detect both the rising and falling edges of the echo signal. This approach improves timing precision and reduces processor workload.

- When the echo signal rises, the timer starts counting.

```
void Ultrasonic::onRise() {  
    _timer.reset();  
    _timer.start();  
}
```

Code Explanation

- `reset()` clears the previous timer value.
- `start()` begins timing the echo pulse duration.
- This ISR executes immediately when the echo signal becomes HIGH.

- When the echo signal falls, the timer stops and the pulse duration is converted into distance.

```
void Ultrasonic::onFall() {  
    _timer.stop();  
  
    _distance = (_timer.elapsed_time().count() * 0.0343f) / 2.0f;  
}
```

Code Explanation

- `stop()` ends the timing process.
- `elapsed_time().count()` obtains the pulse duration.
- 0.0343 represents the speed of sound in cm/μs.
- Division by 2 compensates for the forward and return travel of the ultrasonic wave.

The distance calculation equation used in the project is:

$$d = \frac{t \times 0.0343}{2}$$

Where:

- d = distance in centimeters
- t = echo pulse duration in microseconds

- The interrupt-based approach provides several advantages:

- Higher distance measurement accuracy
- Faster real-time response
- Reduced CPU usage
- Stable sensor operation

Using ISRs significantly improved the performance and responsiveness of the object-following robot system.

8. Implementation

The hardware components were assembled on a robotic chassis. The ultrasonic sensor was mounted at the front side of the robot to detect obstacles directly ahead.

The STM32 Nucleo board was connected to the motor driver and ultrasonic sensor using jumper wires. The embedded code was uploaded to the Nucleo board using the Mbed development environment.

Several tests were conducted to verify:

- Distance measurement accuracy
- Motor response
- Robot stability
- Safe stopping behavior

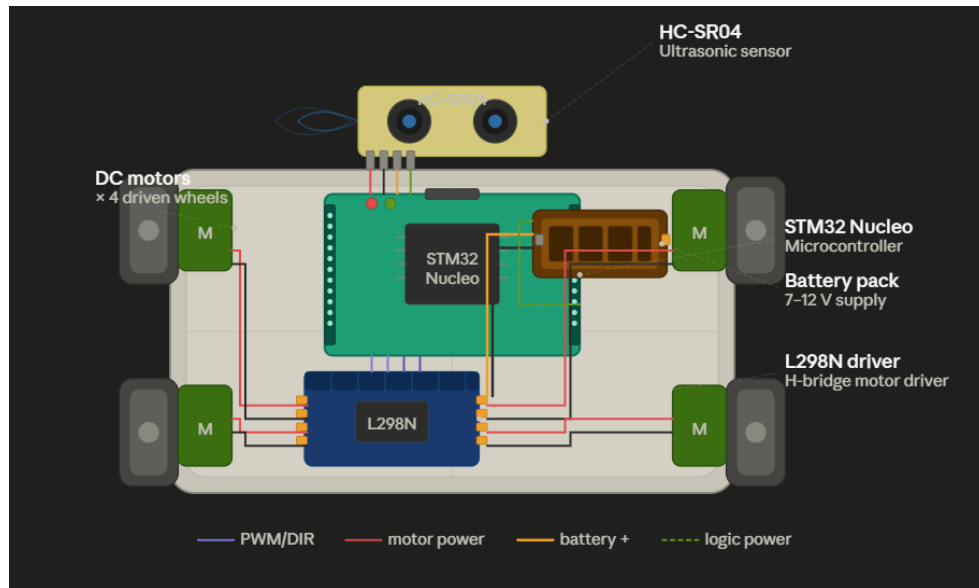


Figure 2: System Integration

9. Results and Testing

The developed robot successfully followed moving objects while maintaining a safe distance. The ultrasonic sensor provided stable distance measurements, and the robot responded correctly according to the implemented control logic.

The testing demonstrated:

- Accurate obstacle detection
- Stable robot movement
- Safe stopping mechanism
- Real-time response capability

The robot was able to:

- Move forward when the object moved away
- Stop at the predefined safe distance

Demo Video Link

- [Implementation Video](#)

10. Advantages of the System

- The developed system provides several advantages:

- Real-time object following
- Simple and low-cost implementation
- Modular software structure
- Safe stopping mechanism
- Efficient hardware utilization

11. Future Improvements

- Possible future improvements include:

- PID control implementation
- Wireless communication
- Obstacle avoidance system
- Bluetooth or Wi-Fi control
- Camera-based tracking
- Speed control optimization

12. Conclusion

This project successfully implemented an embedded robotic system capable of following an object while maintaining a safe distance. The system used an HC-SR04 ultrasonic sensor with a Nucleo development board to perform real-time distance measurements and autonomous movement control.

The project demonstrated practical embedded systems concepts including sensor interfacing, interrupt handling, motor control, and modular programming. The final system achieved stable operation and accurate object-following behavior.

The project also enhanced understanding of embedded software structure, real-time processing, and hardware-software integration in robotic applications.

13. GitHub Repository

- [GitHub Repository Link](#)